

Note

This page was generated from [gallery/plotting_basemap_background.ipynb](#).

Interactive online version: [launch](#) [binder](#)

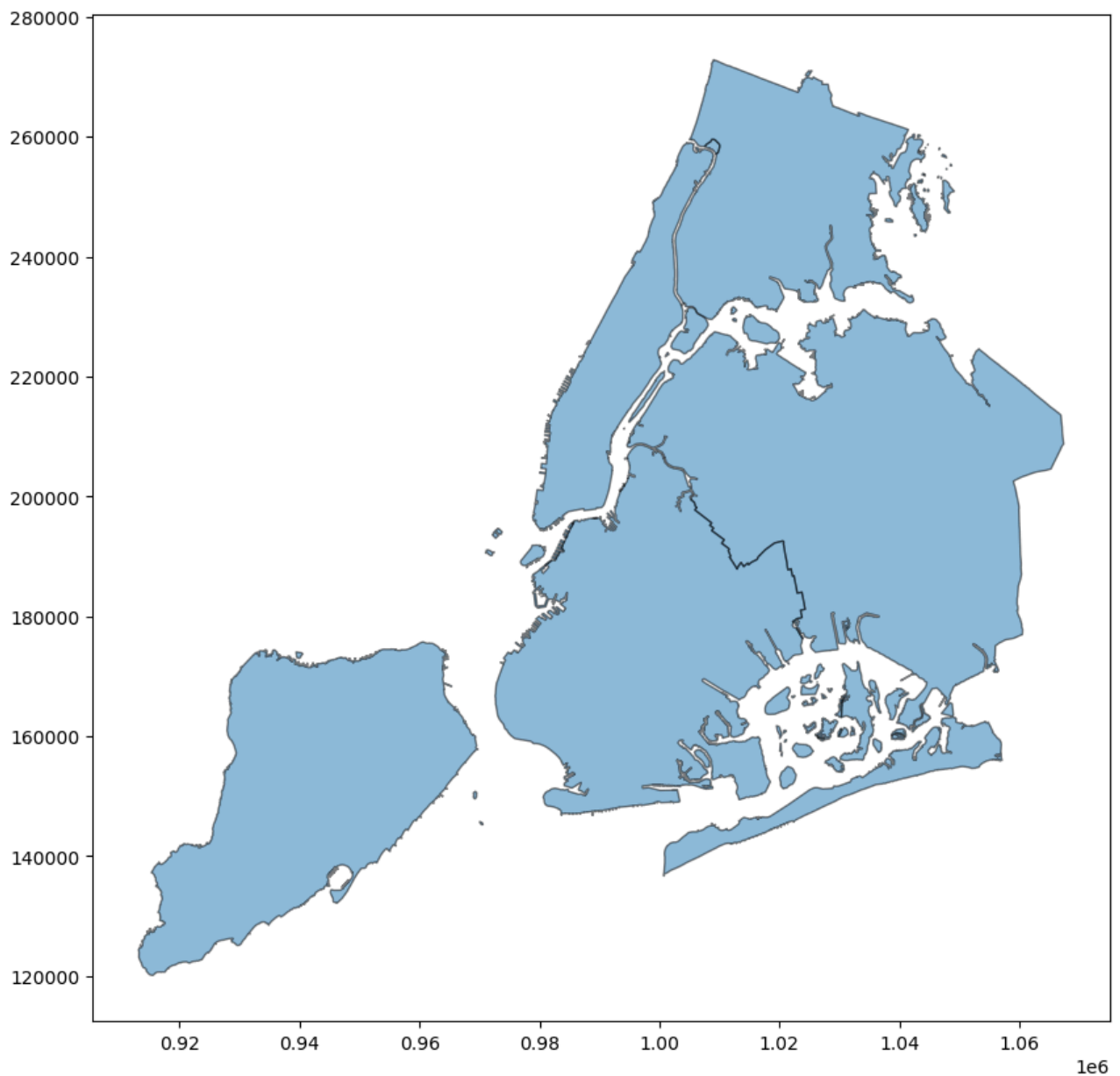
Adding a background map to plots

This example shows how you can add a background basemap to plots created with the geopandas `.plot()` method. This makes use of the [contextily](#) package to retrieve web map tiles from several sources (OpenStreetMap, CartoDB). Also have a look at contextily's [introduction guide](#) for possible new features not covered here.

```
[1]: import geopandas
      import geodatasets
      import contextily as cx
```

Let's use the NYC borough boundary data that is available in geopandas datasets. Plotting this gives the following result:

```
[2]: df = geopandas.read_file(geodatasets.get_path("nybb"))
      ax = df.plot(figsize=(10, 10), alpha=0.5, edgecolor="k")
```



Matching coordinate systems

Before adding web map tiles to this plot, we first need to ensure the coordinate reference systems (CRS) of the tiles and the data match. Web map tiles are typically provided in [Web Mercator \(EPSG 3857\)](#), so let us first check what CRS our NYC boroughs are in:

```
[3]: df.crs
```

```
[3]: <Projected CRS: EPSG:2263>
      Name: NAD83 / New York Long Island (ftUS)
      Axis Info [cartesian]:
      - X[east]: Easting (US survey foot)
      - Y[north]: Northing (US survey foot)
      Area of Use:
      - name: United States (USA) - New York - counties of Bronx; Kings; Nassau; New York;
      - bounds: (-74.26, 40.47, -71.8, 41.3)
      Coordinate Operation:
      - name: SPCS83 New York Long Island zone (US Survey feet)
      - method: Lambert Conic Conformal (2SP)
      Datum: North American Datum 1983
      - Ellipsoid: GRS 1980
      - Prime Meridian: Greenwich
```

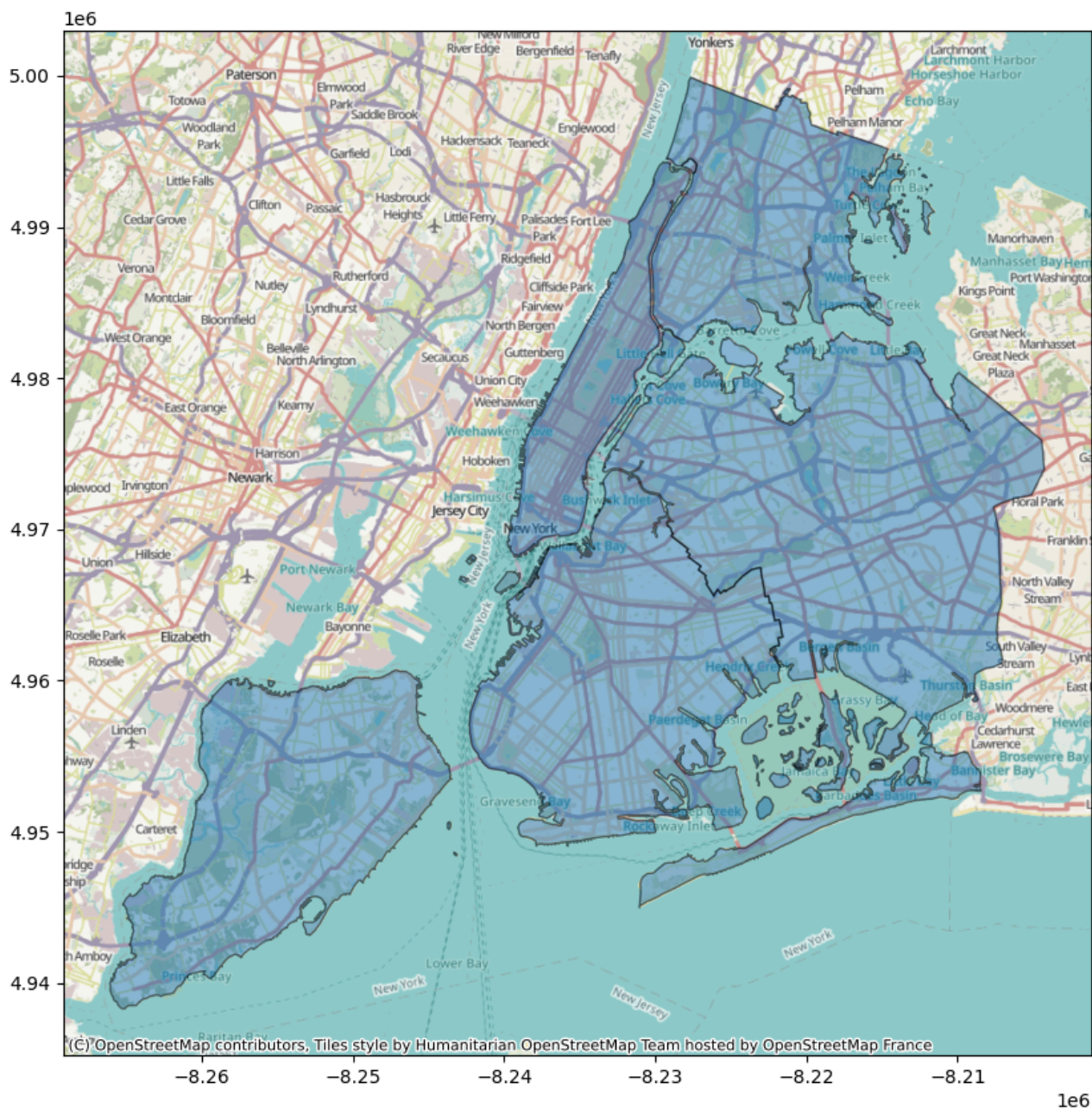
Now we know the CRS do not match, so we need to choose in which CRS we wish to visualize the data: either the CRS of the tiles, the one of the data, or even a different one.

The first option to match CRS is to leverage the `to_crs` method of GeoDataFrames to convert the CRS of our data, here to Web Mercator:

```
[4]: df_wm = df.to_crs(epsg=3857)
```

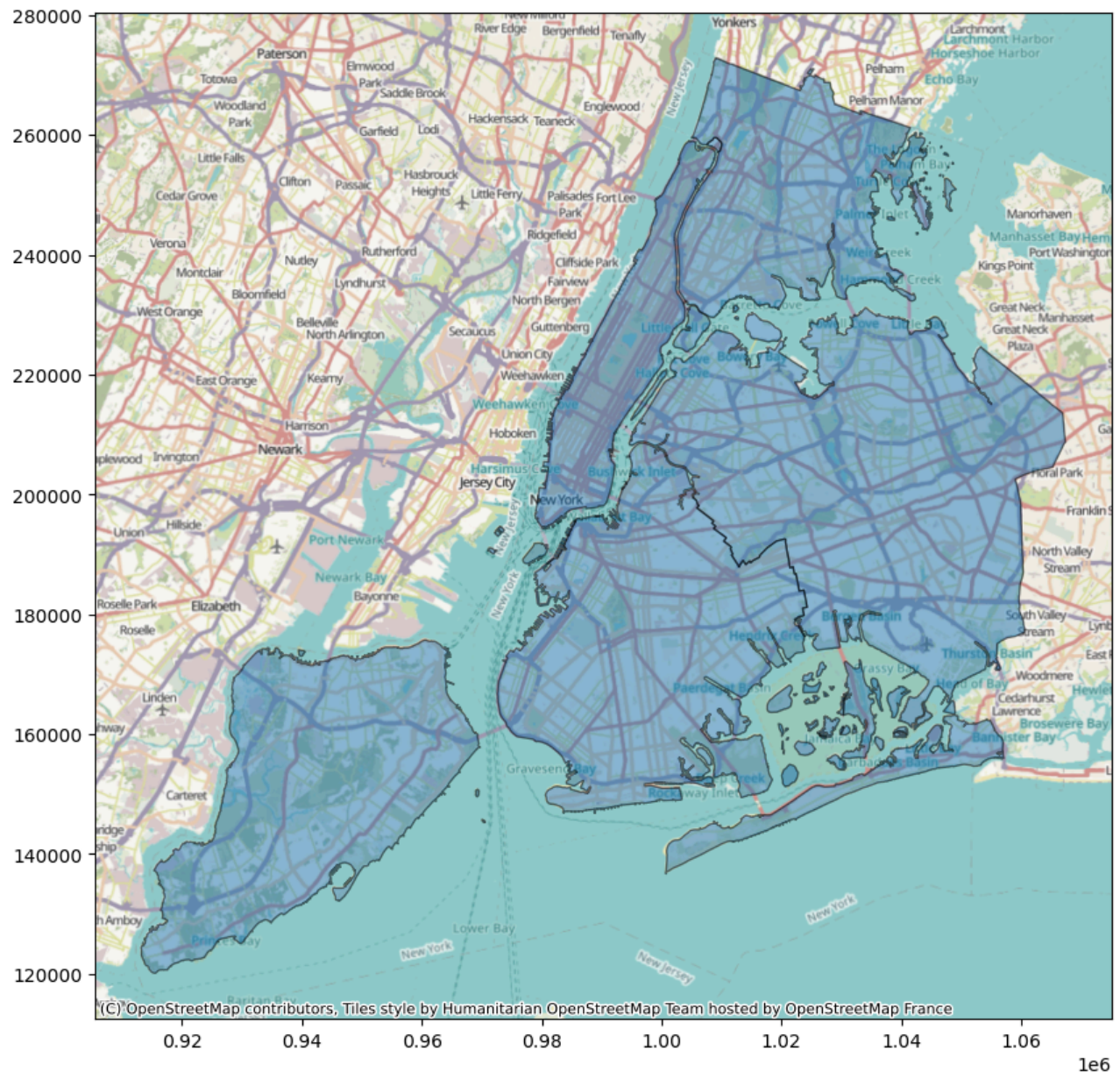
We can then use `add_basemap` function of contextily to easily add a background map to our plot:

```
[5]: ax = df_wm.plot(figsize=(10, 10), alpha=0.5, edgecolor="k")
      cx.add_basemap(ax)
```



If we want to convert the CRS of the tiles instead, which might be advisable for large datasets, we can use the `crs` keyword argument of `add_basemap` as follows:

```
[6]: ax = df.plot(figsize=(10, 10), alpha=0.5, edgecolor="k")
      cx.add_basemap(ax, crs=df.crs)
```

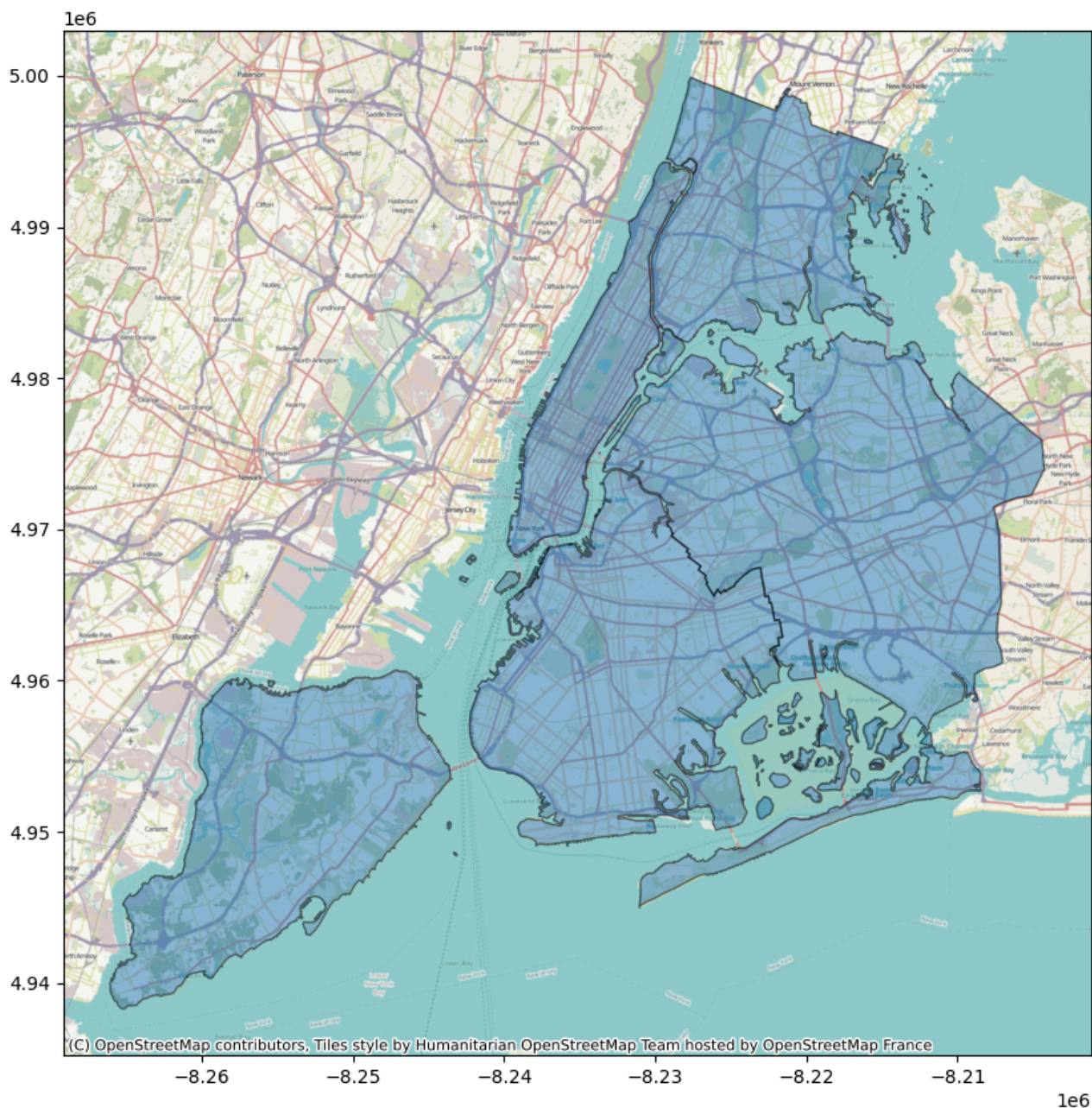



This reprojects map tiles to a target CRS which may in some cases cause a loss of sharpness. See [contextily's guide on warping tiles](#) for more information on the subject.

Controlling the level of detail

We can control the detail of the map tiles using the optional `zoom` keyword (be careful to not specify a too high `zoom` level, as this can result in a large download):

```
[7]: ax = df_wm.plot(figsize=(10, 10), alpha=0.5, edgecolor="k")
     cx.add_basemap(ax, zoom=12)
```

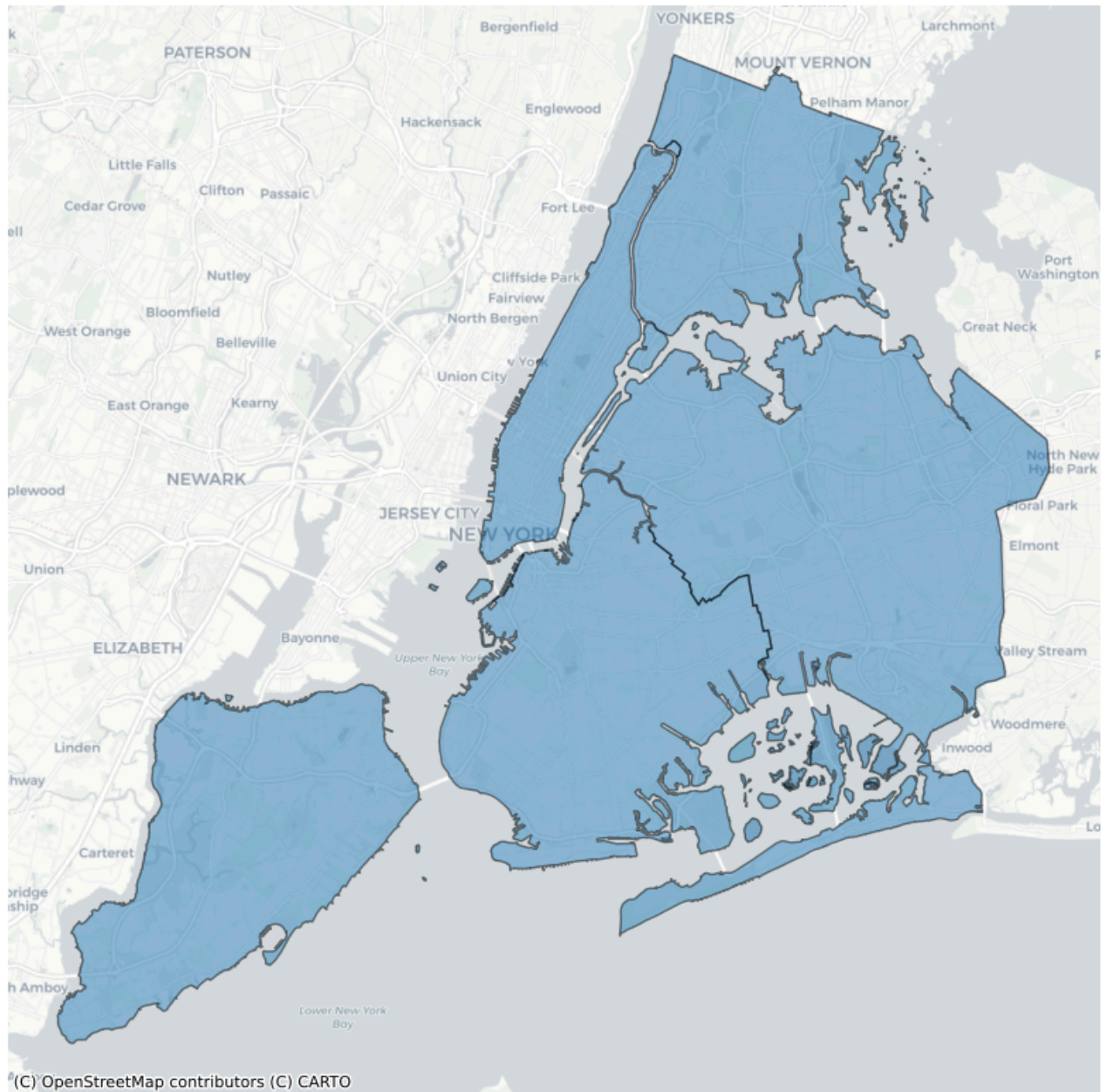


Choosing a different style

By default, contextily uses the OpenStreetMap HOT style. We can specify a different style using

`cx.providers`:

```
[8]: ax = df_wm.plot(figsize=(10, 10), alpha=0.5, edgecolor="k")
      cx.add_basemap(ax, source=cx.providers.CartoDB.Positron)
      ax.set_axis_off()
```

Adding labels as an overlay

© Copyright 2013–, GeoPandas developers.

Created using [Sphinx](#) 7.3.7.

Built with the [PyData Sphinx Theme](#) 0.15.4.